UNIVERSIDAD INTERNACIONAL DEL ECUADOR

Facultad de Ingenierías Digitales y Sociales

Carrera de Ingeniería en Sistemas de Información

Programación Orientada a Objetos

Actividad

Proyecto Integrador Fase 1

Integrantes:

Camila Isabel Paucar Guaranda

Santiago Alejandro Lucina Mena

Pamela Stefania Toapanta Pila

Profesor:

MARÍA DEL CARMEN SALAZAR TORRES

12 de octubre de 2026

Índice

1. ALCANCE, CONTEXTO Y PROBLEMA DE LA EMPRESA ----------2
2. REQUISITOS MÍNIMOS ----------3
3. FUNDAMENTOS DE PROGRAMACIÓN ORIENTADA A OBJETOS EN GO----------4
4. DIAGRAMA DE GANTT Y CRONOGRAMA DE TRABAJO----------4
5. MATRIZ DE ACTORES, ROLES Y RESPONSABILIDADES ----------5
6. CONTRATO DE COMUNICACIÓN DEL EQUIPO ----------7
7. DIAGRAMA DE CLASES UML----------8
8. DECLARACIÓN DE USO DE INTELIGENCIA ARTIFICIAL Y CRITERIO PROPIO----------9
9. INTEGRIDAD ACADÉMICA Y LICENCIA DEL PROYECTO ---------- 10
10. REPOSITORIO DEL PROYECTO ---------- 10

1. ALCANCE, CONTEXTO Y PROBLEMA DE LA EMPRESA

El alcance comprende el diseño e implementación de una aplicación cliente-servidor desarrollada modularmente en Go para la gestión de correos internos entre las dos sedes, la cual operará sobre la infraestructura de red interconectada por OSPF e integrada con un servicio de autenticación en FastAPI. El sistema abarcará las funciones esenciales de redacción, envío, consulta de bandejas de entrada y salida, lectura con actualización de estado y eliminación de mensajes entre usuarios autenticados, aplicando principios de modularidad, encapsulamiento y control de acceso por tokens en cada transacción. Se excluye explícitamente la integración con protocolos o servidores SMTP externos, el soporte para archivos adjuntos pesados y esquemas de alta disponibilidad o balanceo de carga, limitándose el servicio a la mensajería interna controlada de la organización.

Planteamiento del Problema:

Con la apertura de su segunda sede, la empresa enfrenta un problema de comunicación. Esto ha generado una pérdida de información al momento de dar instrucciones y también ha habido retrasos en la comunicación al depender de canales informales o aplicaciones externas sin ningún control. La falta de un canal centralizado dificulta el seguimiento de las decisiones administrativas entre ambas oficinas.

La implementación de un sistema de gestión de correos internos resuelve esta necesidad de tener un canal oficial, seguro y ordenado. Más allá de ser una opción accesible, garantiza que la comunicación entre sedes quede registrada, permitiendo además el análisis de datos cuantitativos medibles para el área de Estadística, estos incluyen el volumen diario de correos enviados y tiempos de entrega. Esto lo hace sin comprometer la seguridad ni sobrecargar la red.

2. REQUISITOS MÍNIMOS

- **RF-SW-01: Arquitectura Cliente-Servidor:** Aplicación desarrollada en Go con el cliente y el servidor completamente desacoplados para operar entre las sedes a través de la red, garantizando que el cliente envíe peticiones TCP/HTTP al servidor y reciba respuestas estructuradas sin que exista un acoplamiento directo de código.
- **RF-SW-02: Gestión de Correos Internos:** Sistema que incorpora las funcionalidades esenciales de redacción, envío, consulta de bandejas, lectura con cambio de estado a *leído* y eliminación de mensajes, logrando que al consultar la bandeja los mensajes no leídos se distingan claramente de los ya leídos.
- **RF-SW-03: Integración con Autenticación:** Mecanismo encargado de interceptar y validar en cada transacción de correo el token de sesión emitido

por el servicio de FastAPI, asegurando que, si el token es inválido o no existe, el servidor deniegue la transacción de forma inmediata.

- **RF-SW-04: Modularidad y Encapsulamiento:** Organización estricta del proyecto en paquetes independientes que separan la red, el negocio y el almacenamiento, asegurando que los paquetes compilen de forma aislada y respeten rigurosamente las normas de visibilidad de Go entre elementos exportados y no exportados.
- **RF-SW-05: Manejo Robusto de Errores:** Sistema de control y retorno explícito de errores en Go diseñado para prevenir fallos críticos o caídas del servidor, garantizando que ningún error generado por una entrada inválida del usuario provoque un cierre forzoso.
- **RF-SW-06: Concurrencia:** Mecanismo de manejo concurrente en el servidor mediante el uso de *goroutines*, permitiendo atender solicitudes simultáneas y paralelas de múltiples clientes provenientes de ambas sedes de forma segura a través de canales.

3. FUNDAMENTOS DE PROGRAMACIÓN ORIENTADA A OBJETOS EN GO

Es importante tomar en cuenta que al momento de desarrollar este software en Go, los conceptos habituales de la Programación Orientada a Objetos se adaptan a las características nativas del lenguaje:

- **Clases y Encapsulamiento:** En lugar de usar de clases habituales, se utilizan estructuras (*structs)* con sus funciones asociadas. El encapsulamiento se logra usando nombres con mayúsculas o minúsculas para ocultar o mostrar datos, y separando el proyecto en paquetes de red, negocio y datos.
- **Composición en lugar de Herencia:** Go no utiliza una herencia común, en su lugar la reutilización de código y la jerarquía se resuelven mediante la composición, incrustando un *struct* dentro de otro.
- **Polimorfismo mediante Interfaces:** El polimorfismo se logra implementando interfaces. Un ejemplo claro es la interfaz RepositorioCorreo, la cual define las acciones básicas para guardar datos. Esto permite que el sistema pueda cambiar su forma de almacenamiento de forma flexible, aislando por completo la lógica de negocio y evitando alterar el funcionamiento principal de la aplicación.

4. DIAGRAMA DE GANTT Y CRONOGRAMA DE TRABAJO

DIAGRAMA DE GANTT																	
FASE	ACTIVIDADES	SEMANAS															
		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
FASE 1: CONTEXTO, PLANIFICACIÓN Y ORGANIZACIÓN	Documento de contexto (problema y necesidad) y selección del sistema																
	Matriz de actores, roles y responsabilidades																
	Repositorio Git inicial con README y licencia																
	Video breve de presentación y distribución de roles																
FASE 2: ANÁLISIS Y REQUISITOS	Requisitos funcionales de correo en Go, alcance, restricciones y exclusiones																
	Casos de uso y flujos (redactar, enviar, leer, borrar)																
	Video explicativo de análisis y alcance de software																
FASE 3: DISEÑO DE LA SOLUCIÓN	Arquitectura cliente-servidor de la aplicación Go																
	Modelo de datos (FastAPI) y políticas de autorización																
	Diagramas de paquetes/clases, módulos y concurrencia Go																
	Diseño de interfaz, accesibilidad y prototipo visual																
FASE 4: PROTOTIPO Y DESARROLLO INICIAL	App Go: estructura modular, servidor, cliente, operaciones principales y validaciones																
	Repositorio actualizado con estructura modular Go																
	implementación de concurrencia con goroutines para la atención de múltiples clientes																
	Implementación de concurrencia y canales con goroutines en la aplicación Go.																
	Código inicial funcional de la aplicación Go																
	Resultados de la prueba piloto y ajustes																
	Video de avance y demostración parcial.																
FASE 5: INTEGRACIÓN Y DESPLIEGUE	Versión integrada de la aplicación Go con FastAPI																
	Pruebas de integración, solución de errores y evidencia																
	Registro de métricas preliminares (latencia/tiempos)																
	Evidencias de autenticación desde la aplicación Go.																
FASE 6: PRUEBAS, VALIDACIÓN ESTADÍSTICA Y EVALUACIÓN	Casos de prueba funcionales/errores (manejo de fallos)																
	Exportar datos cuantitativos y matriz de resultados																
	Versión candidata final																
	Video demostrativo de pruebas y software																
FASE 7: PRESENTACIÓN FINAL Y DEFENSA	Documento final (Informe)																
	Presentación académica y demostración en vivo																
	Video demostrativo final y Defensa oral																
FASE 8: CIERRE, CORRECCIONES Y ENTREGA DEFINITIVA	Corrección de errores de defensa y Acta de cambios																
	Manual de instalación, configuración y ejecución Go																
	Actualización final del repositorio y versión definitiva del sistema en Go																

Leyenda de Estados:

Completado No iniciado No completado

5. MATRIZ DE ACTORES, ROLES Y RESPONSABILIDADES

A. Actores

Actor	*Actividad en el Sistema*	*Riesgo Asociado*	*Consideraciones Éticas*	*Posible Solución*
Usuario General	Redactar, enviar, consultar bandejas de entrada/salida y leer correos corporativos internos.	Suplantación de identidad por descuido de credenciales o dejar la sesión abierta.	Garantizar la confidencialidad y privacidad de la correspondencia entre los empleados.	Implementar validación estricta de tokens de sesión con FastAPI y un diseño de interfaz intuitivo.
Administrador de Red y Sistemas	Configurar los routers, enlaces OSPF, VLANs y supervisar el servicio de autenticación FastAPI.	Abuso de privilegios para acceder a bandejas ajenas o interrupción de enlaces entre sedes.	Obligación estricta de no vulnerar la privacidad de los mensajes institucionales.	Aplicar el principio de mínimo privilegio y auditoría en el acceso a los servidores de datos.
Directiva Institucional	Definir políticas de comunicación interna y supervisar la operatividad general del sistema entre sedes.	Pérdida de información administrativa crítica ante fallas en la red o caída del servidor.	Alta responsabilidad ética en la custodia de los datos y la toma de decisiones institucionales.	Realizar respaldos periódicos de la persistencia de datos y asegurar los canales de transmisión.

B. Usuarios Iniciales de Prueba

Identificador de Usuario (Username)	*Sede / Ubicación en la Red*	*Rol Principal*	*Funciones Asignadas para Pruebas*

usr_sedeA_ventas	Sede A (Red de Usuarios / VLAN 10)	Emisor / Usuario Operativo	Redacción de correos institucionales, envío de solicitudes intermitentes y consulta de bandeja de salida.
usr_sedeB_admin	Sede B (Red de Servidores / VLAN 20)	Receptor / Usuario Administrativo	Consulta de bandeja de entrada, lectura de mensajes con actualización automática a "leído" y eliminación lógica de registros.
admin_red	Nodo Central / Administrador	Supervisor Técnico	Validación de tokens de sesión mediante FastAPI y monitoreo del flujo de red interconectado por OSPF.

C. Roles y Responsabilidades del Equipo de Trabajo

Integrante del Equipo	***Responsabilidades en la Fase 1***	***Responsabilidades en Fases Posteriores***
Camila Isabel Paucar Guaranda	Análisis del problema organizacional, definición de la justificación del sistema y estructuración de la matriz de actores y consideraciones éticas.	Desarrollo de los paquetes de red y lógica de comunicación cliente-servidor en Go, e integración con las rutas de FastAPI.
Santiago Alejandro Lucina Mena	Diseño de los requerimientos funcionales de software (RF-SW), modelado de POO en Go y redacción de declaraciones formales.	Implementación de la persistencia de datos, gestión de concurrencia mediante *goroutines* y pruebas de sobrecarga en la red.
Pamela Stefania Toapanta Pila	Elaboración del cronograma de trabajo, configuración del diagrama de Gantt y definición de las políticas de integridad y licencias.	Diseñar e implementar una infraestructura de red en dos sedes mediante enrutadores OpenWrt, segmentación VLAN, enrutamiento OSPF, protocolos de alta disponibilidad (RSTP, EtherChannel), direccionamiento VLSM sobre el bloque 10.10.0.0/16, y servicios de DHCP, NAT y firewall.

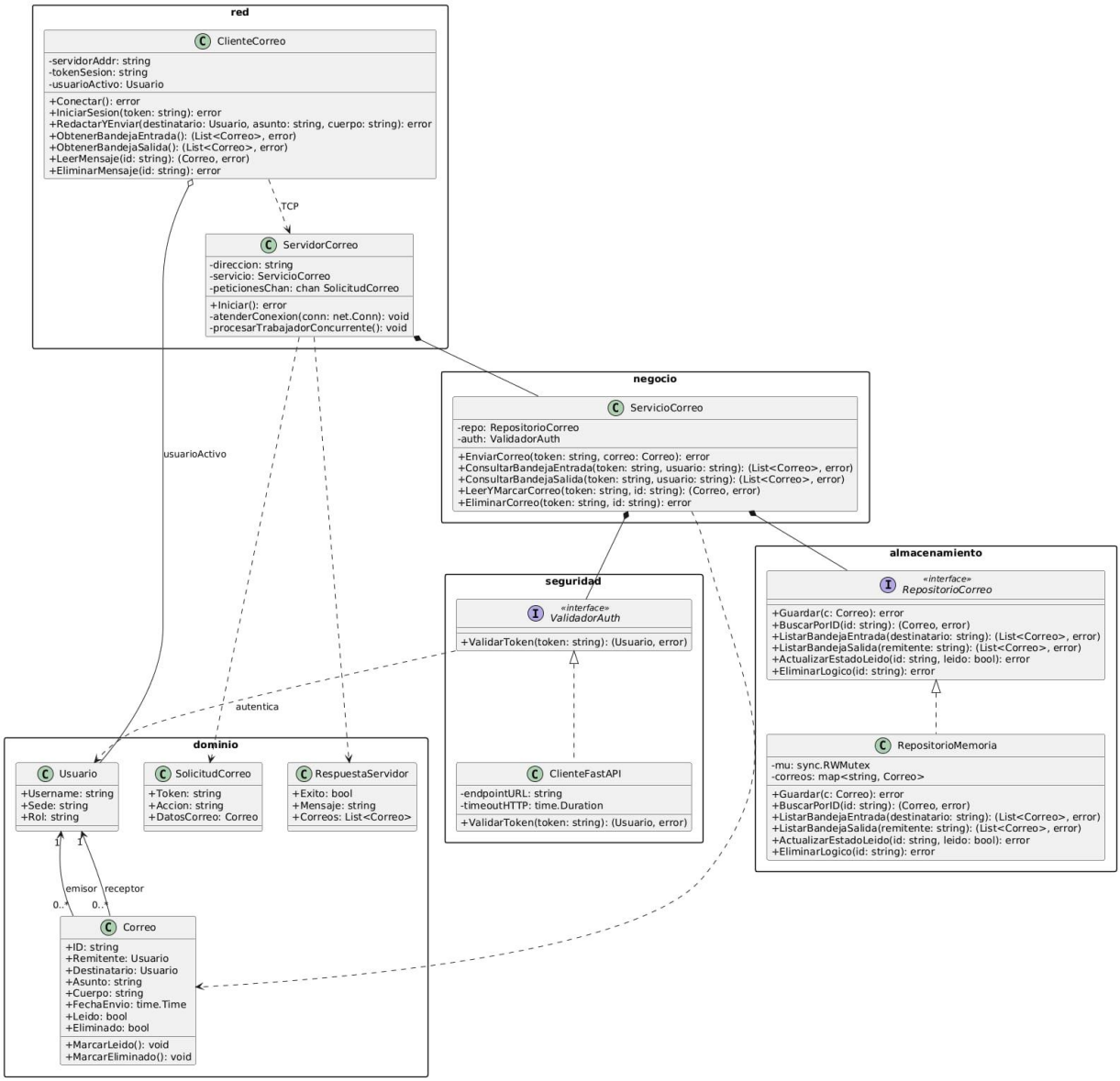
6. CONTRATO DE COMUNICACIÓN DEL EQUIPO

Para asegurar un flujo de trabajo fluido y un rendimiento óptimo durante el desarrollo del proyecto, hemos establecido diferentes normas y acuerdos que se encuentran a continuación:

- **Canales Oficiales:** El canal principal de coordinación diaria será WhatsApp. En caso de requerir revisiones de código en tiempo real, o explicaciones técnicas que necesiten soporte visual, se utilizará Microsoft Teams como plataforma para reuniones virtuales y compartición de pantalla.
- **Tiempos de Respuesta:** Se establece un compromiso por parte de todos los integrantes de procurar un plazo de respuesta estimado entre **15 y 45 minutos** para mensajes o consultas importantes enviadas a través del canal oficial, garantizando así la agilidad en la toma de decisiones.
- **Toma de Decisiones y Resolución de Conflictos:** Si se llega a presentar un desacuerdo, el proceso a seguir será el siguiente:
    - Se expondrá el problema y las alternativas propuestas por los integrantes, es decir nosotros.
    - Cada miembro aportará su perspectiva o solución para abrir un espacio de debate constructivo.
    - Finalmente, se realizará una votación, adoptando e implementando la idea que obtenga la mayoría de los votos por considerarse la más acertada para el cumplimiento de los objetivos del proyecto.

7. DIAGRAMA DE CLASES UML

Figura 1. Diagrama de clases uml



Nota. Elaboración propia

8. DECLARACIÓN DE USO DE INTELIGENCIA ARTIFICIAL Y CRITERIO PROPIO

Durante el desarrollo de esta fase del proyecto, se utilizó la herramienta de inteligencia artificial *Gemini* como un recurso de apoyo y consulta. Su función principal fue asistir al equipo en el análisis comparativo de las diferentes opciones de aplicaciones empresariales propuestas, evaluando los beneficios, las limitaciones técnicas y la viabilidad de implementación bajo una arquitectura cliente-servidor en Go.

Prompts Utilizados:

a. **Consulta inicial de alcance y selección:**

"En base al documento, ayúdame a definir el alcance para la materia de Programación Orientada a Objetos en las Fases 1 y 2, y recomiéndame cuál de las opciones de aplicación empresarial es la más fácil de desarrollar."

b. **Apoyo en la estructuración de la arquitectura y diseño:**

"Ayúdame a estructurar la arquitectura del sistema en Go y explícame cómo el uso de interfaces permite separar la lógica de negocio del almacenamiento de datos."

Criterio propio aplicado:

Como decisión del equipo, no adoptamos la recomendación de la IA de forma automática. Analizamos las opciones y descartamos aquellas que requerían herramientas externas muy pesadas. Elegimos el sistema de gestión de correos internos porque se ajusta perfectamente a los conocimientos que hemos adquirido a lo largo del semestre. Esto nos permite hacer una arquitectura limpia, aplicar los principios de POO en Go y sacar métricas estadísticas reales para las siguientes fases del proyecto.

9. INTEGRIDAD ACADÉMICA Y LICENCIA DEL PROYECTO

Los integrantes del equipo confirmamos que este trabajo ha sido realizado por nuestra cuenta, cumpliendo con los principios de honestidad académica y las normativas de la Universidad Internacional del Ecuador. Las fuentes de información y herramientas de apoyo externas han sido debidamente declaradas en este documento.

Este proyecto se distribuye bajo la Licencia MIT, una licencia de código abierto permisiva que permite a cualquier usuario utilizar, modificar, copiar y distribuir el software de manera libre, siempre y cuando se mantenga el aviso de autoría y los créditos correspondientes a los desarrolladores originales.

10. REPOSITORIO DEL PROYECTO

Enlace de GitHub:

- https://github.com/Slucina04060708/PROYECTO-INTEGRADOR